SBD UD1 · Fundamentos de Big Data (ampliación)

José Manuel Sánchez Álvarez - IES Rafael Alberti

Índice de contenidos

UD1 · Fundamentos de Big Data — capítulo ampliado **1**
1. Punto de partida: la promesa y el problema . . . 1
2. Qué es Big Data (y qué no es) . . . 2
3. Por qué ahora: una convergencia . . . 2
4. Las “5 V” como manera de pensar . . . 2
5. Tipologías de datos: estructura, latencia y sensibilidad . . . 3
6. Formatos: por qué Parquet gana al CSV (y cuándo usar Avro o JSON) . . . 3
7. Data warehouse, data lake y la idea de “lakehouse” . . . 3
8. ETL o ELT: el orden de los factores sí altera el coste . . . 4
9. Un caso pequeño para amarrar ideas . . . 4
10. Coste, calidad y tiempo: tres cuerdas que tensar . . . 4
11. Errores frecuentes y antídotos . . . 5
12. Qué viene a continuación en la UD1 . . . 5

UD1 · Fundamentos de Big Data — capítulo ampliado

1. Punto de partida: la promesa y el problema

Cada organización, desde un instituto hasta una multinacional, genera y recibe datos sin parar: ventas, visitas a una web, encuestas, sensores, documentación, imágenes, correos... La promesa del llamado *Big Data* es convertir ese ruido en decisiones mejores y más rápidas: planificar personal, ajustar stock, detectar fraude, optimizar rutas, personalizar contenidos, anticipar la demanda. El problema es que los datos ya no caben en una sola hoja de cálculo ni obedecen a un único formato. No llegan todos al mismo ritmo, no tienen la misma calidad y, sobre todo, no cuestan lo mismo de almacenar y procesar. Big Data no es un eslogan: es el nombre de un conjunto de prácticas y tecnologías que hacen viable extraer valor de datos **grandes**, **rápidos** y **variados** sin arruinarnos por el camino.

En esta unidad usaremos a menudo un caso sencillo: una cadena minorista que recibe ventas desde tiendas físicas en CSV y pedidos de comercio electrónico en JSON. El caso parece pequeño, pero contiene muchos problemas reales: retrasos de carga, códigos de producto escritos de formas distintas, duplicados al reenviar ficheros y dudas sobre qué sistema contiene la versión correcta de cada dato. Un **SKU** (*Stock Keeping Unit*) es el código que identifica un producto concreto. Si un mismo producto aparece como `ABC-123`, `abc123` y `ABC123`, el sistema puede contarlo como tres productos diferentes. Ahí empieza la necesidad de limpiar, normalizar y apoyarse en un **catálogo maestro** de productos.

2. Qué es Big Data (y qué no es)

Llamaremos Big Data a la capacidad de **obtener valor accionable** a partir de datos cuyo **volumen**, **velocidad** o **variedad** desbordan las herramientas tradicionales, y que exigen enfoques de **procesamiento y almacenamiento coste-eficientes**. No es solo "datos enormes" ni "poner Hadoop/Spark y ya está". Tampoco es únicamente inteligencia artificial. Big Data empieza **cuando los límites prácticos** de nuestras herramientas habituales (RAM, CPU de una sola máquina, tiempos de espera razonables) nos obligan a distribuir el almacenamiento y/o el cómputo, a elegir formatos más inteligentes y a automatizar el movimiento de datos con métodos reproducibles.

Tampoco conviene medirlo solo en gigabytes. Puede haber Big Data por **volumen** (demasiados datos para una máquina), por **velocidad** (llegan continuamente y el valor caduca rápido), por **variedad** (mezcla de CSV, JSON, logs, APIs y tablas) o por la combinación de todo ello con exigencias de **calidad** y **coste**. En formación trabajaremos con datasets manejables, pero simulando decisiones reales: qué guardo como raw, qué limpio, qué formato uso y cómo justifico que el dato ya es fiable.

3. Por qué ahora: una convergencia

Hay tres fuerzas que explican el auge del Big Data. Primero, la **digitalización** masiva: casi todo lo que hacemos deja huella en forma de evento, log o registro. Segundo, la **caída del precio** del almacenamiento y el cómputo: clústeres con hardware "normalito" y la nube han hecho accesible lo que antes era exclusivo. Tercero, el **ecosistema abierto**: proyectos como Spark, Kafka o sistemas columnares (Parquet, ORC) resolvieron cuellos de botella y democratizaron la analítica a gran escala. El resultado es un terreno fértil: hoy es razonable plantearse preguntas complejas con datos verdaderamente voluminosos y obtener respuestas en tiempos útiles para el negocio.

4. Las "5 V" como manera de pensar

Hablar de Big Data sin las "5 V" es como hablar de cocina sin ingredientes. No son una definición legal, sino una **brújula** para decidir arquitecturas y costes.

Volumen. Imagina años de ventas de un retail a nivel de ticket, más logs de navegación y devoluciones. El volumen no es solo el tamaño total en disco: influye en cómo particionas los datos, en cuántos ficheros produces y en si un *join* cabe o no en memoria.

- **Velocidad.** Hay casos en los que los datos llegan como un goteo continuo (sensores, clics, transacciones). Procesar al vuelo o por micro-lotes importa porque el valor caduca: detectar un pico de tráfico hoy vale; mañana, es historia.
- **Variedad.** CSVs, JSON jerárquicos, tablas relacionales, imágenes, texto libre. La variedad nos obliga a **normalizar**, validar y traducir formatos para hacer posible el análisis conjunto.
- **Veracidad.** Sin calidad, no hay decisión fiable. Nulos, duplicados, errores de formato o inconsistencias entre columnas pueden anular un KPI brillante. La veracidad exige reglas explícitas de validación y corrección.
- **Valor.** La V más olvidada y la más decisiva. Todo lo anterior tiene sentido si termina en **acciones**: menos roturas de stock, menos abandono de clientes, más eficiencia operativa.
- **Variabilidad.** A veces se añade como sexta V. Recuerda que los datos no siempre se comportan igual: hay campañas, picos de demanda, cambios de formato, productos nuevos, estacionalidad y errores que aparecen solo en determinadas fuentes. La variabilidad obliga a revisar reglas y métricas, no a darlas por cerradas para siempre.

5. Tipologías de datos: estructura, latencia y sensibilidad

Cuando pensamos *qué datos tenemos*, conviene clasificarlos en tres dimensiones. **Estructura.** Los datos **estructurados** viven en tablas con tipos claros y claves (ventas, alumnos, facturas). Los **semiestructurados** (JSON, XML) mezclan jerarquías y listas; son flexibles para APIs y eventos. Los **no estructurados** (texto, imagen, audio, vídeo) requieren técnicas específicas para ser "analizables". **Latencia.** Algunos datos exigen respuesta en **tiempo real** (milisegundos o segundos), otros permiten **lote** (minutos, horas, D+1). La latencia determina tecnologías y costes: no es lo mismo apuntar una alerta inmediata que preparar un informe mensual. **Sensibilidad.** No todo pesa lo mismo en términos legales y éticos. Identificadores personales, salud o finanzas requieren otro nivel de control que datos anónimos o públicos. Esta dimensión conecta con RGPD, gobierno y acceso.

6. Formatos: por qué Parquet gana al CSV (y cuándo usar Avro o JSON)

El formato es una decisión de coste y rendimiento camuflada de detalle técnico. **CSV** es universal y humano, pero **caro** para analítica: cada lectura implica parsear texto y no permite saltar a columnas concretas. **JSON** es ideal para intercambio y APIs; su jerarquía es expresiva, pero también **pesada** de procesar en grandes volúmenes. **Parquet** y **ORC** almacenan por **columnas**, comprimen muy bien y permiten *predicate pushdown*: si solo necesitas `importe` de junio, no lees el resto. Parquet se ha convertido en la **opción por defecto** para zonas *curated* de un *data lake*. **Avro** almacena por filas y convive muy bien con **streaming** y **Kafka** gracias al *Schema Registry*. Es apropiado cuando importa preservar el esquema versión a versión y la escritura es continua. En resumen: **CSV/JSON para ingesta e intercambio**, **Parquet para analítica**, **Avro** para flujos de eventos y contratos de esquema.

Algunas expresiones técnicas aparecen mucho en documentación profesional:

Concepto	Idea esencial
Formato columnar	Guarda juntas las columnas, no las filas completas. Es eficiente cuando analizas pocas columnas de muchas filas.
Compresión	Reduce tamaño en disco y a menudo también tiempo de lectura, porque se mueve menos dato.
Predicate pushdown	El motor evita leer datos que no cumplen el filtro de la consulta. Si pides `mes = 6`, no necesita leer todos los meses.
Schema Registry	Registro de versiones de esquemas para que productores y consumidores de eventos sepan qué estructura tiene cada mensaje.

7. Data warehouse, data lake y la idea de "lakehouse"

Un **data warehouse** tradicional prioriza estructura, gobernanza y rendimiento en SQL empresarial; obliga a transformar y cargar datos ya "limpios" y bien modelados. Un **data lake** es barato y flexible: puedes aterrizar datos casi crudos, de muchos tipos, y procesarlos después. El **lakehouse** intenta unir lo mejor de ambos: mantener datos en formatos de *lake* (Parquet) con **capas transaccionales** que ofrecen *ACID*, *time travel* y *upserts* (ej. Delta Lake, Iceberg, Hudi). En la práctica educativa, basta con entender que el warehouse es el salón ordenado, el lake es el trastero enorme, y el lakehouse es el trastero ordenado con estanterías etiquetadas.

Cuando decimos **ACID** hablamos de garantías clásicas de las bases de datos: atomicidad, consistencia, aislamiento y durabilidad. Dicho de forma práctica: operaciones que se completan enteras o no se completan, datos que no quedan a medias y cambios que sobreviven a fallos. En un lakehouse, tecnologías como Delta Lake, Iceberg o Hudi intentan llevar parte de esas garantías a ficheros Parquet almacenados en un data lake. No necesitamos dominarlas ahora; nos interesa saber qué problema resuelven.

8. ETL o ELT: el orden de los factores sí altera el coste

En **ETL** transformas antes de cargar: útil cuando el destino es rígido y caro (un warehouse con licencias y reglas estrictas). En **ELT** cargas primero al *lake* y transformas después, aprovechando cómputo elástico y barato. Para este módulo trabajaremos mentalmente en **ELT**: aterrizamos en **raw**, estandarizamos en **processed** y publicamos en **curated**, casi siempre en **Parquet** y con **particionado temporal** (año, mes, día). Esta estrategia simplifica el re-procesado, abarata experimentación y deja huella clara del linaje.

Las palabras **raw**, **processed** y **curated** no son marcas comerciales, sino etiquetas útiles para pensar por capas. *Raw* conserva lo que llegó. *Processed* contiene datos ya tipados o normalizados. *Curated* es la versión preparada para consulta, informes o modelos, con reglas de calidad aplicadas y decisiones documentadas.

9. Un caso pequeño para amarrar ideas

Pensemos en la cadena minorista del inicio. Tiene un CSV diario con ventas de tiendas físicas y una API que devuelve pedidos web en JSON. Ambos sistemas hablan de productos, pero no siempre con el mismo formato de SKU. Además, un fichero de tienda puede reenviarse y crear duplicados.

El flujo razonable es este, en prosa: el CSV y el JSON aterrizan tal cual en **raw** para no perder el original. Después se normalizan tipos (fechas, enteros, decimales), se limpian espacios y mayúsculas en `sku`, se valida cada producto contra el catálogo maestro y se detectan duplicados por clave natural, por ejemplo `fecha + tienda + sku`. Con ambas fuentes controladas, se construye un **conjunto curado** en Parquet, particionado por fecha y quizá por canal.

A partir de ahí se calculan **KPIs** sencillos: ventas por día, unidades por canal, productos con rotura de stock y diferencias entre tienda física y web. La decisión práctica surge sola: si un producto vende mucho en web pero aparece sin stock en tiendas, dirección puede revisar reposición. No hace falta "predicción mágica" para crear valor: basta con **datos veraces, oportunos y bien modelados**.

10. Coste, calidad y tiempo: tres cuerdas que tensar

El triángulo *coste–calidad–tiempo* gobierna cada elección. Un ejemplo: si dejamos todo en CSV para "no complicarnos", el almacenamiento parece barato, pero la **lectura** y las uniones se vuelven lentas y frágiles; el tiempo de espera sube y acabamos "costando" más en horas y errores. Otro ejemplo: declarar una columna `precio` como texto obliga a reconversiones y arruina la compresión; declararla como `double` desde el principio ahorra tamaño y acelera filtros. Decidir **particionado por fecha** permite que una consulta de una semana no lea años completos. Son decisiones pequeñas con impacto multiplicado.

11. Errores frecuentes y antídotos

El primer error es **coleccionar** datos sin plan de uso: acumular gigas no es una estrategia. Debe existir una **pregunta de negocio** que haga de brújula. El segundo es **idealizar** el tiempo real: la mayoría de decisiones aceptan **D+1**; perseguir latencias milimétricas sin necesidad eleva exponencialmente la complejidad. El tercero es **subestimar la calidad**: un KPI espectacular construido sobre duplicados o fechas mal tipadas es humo. Validar dominios, rangos y consistencia entre columnas es tan importante como el modelo visual que lo mostrará. Y el cuarto, muy común, es **mantener CSV eternos** en producción: úsalo para intercambiar, pero persiste tu capa **curated** en **Parquet** y mide la mejora.

12. Qué viene a continuación en la UD1

Ahora que tenemos el mapa, nos meteremos en la mecánica: **EDA y calidad de datos** (qué mirar y cómo decidir), **tipado y limpieza**, **exportación eficiente a Parquet** y **primeras consultas interactivas con DuckDB**. Veremos el impacto de pasar un dataset de CSV a Parquet en tamaño y velocidad, y cómo documentar, con cabeza, las decisiones que afectan a la veracidad y al coste.